Logging and Observability - Cheat Sheet

Observability is how you understand what your system is doing from the outside. Logs, metrics, and traces are the standard tools. Each answers a different question: what happened (logs), how much or how often (metrics), where did the time go (traces).

A system without good observability isn't broken until it is, and then you're guessing in the dark.

The three pillars

The classic framing. Each pillar covers what the others miss.

Pillar	Question it answers	Best at
Logs	What happened at time T?	Discrete events, error context
Metrics	How much, how often, how fast?	Aggregates, trends, alerts
Traces	Where did the time go in this request?	Cross-service latency breakdown

Recent thinking adds events and continuous profiles as additional signals, but those three are still the foundation.

Logging

Frameworks in the Java ecosystem

Framework	Role
SLF4J	Facade. The interface you code against.
Logback	Implementation. The default in Spring Boot.
Log4j2	Alternative implementation. Better async performance.
`java.util.logging` (JUL)	JDK's built-in. Rare in modern apps.

Always code against SLF4J, even when Logback is what's behind it. Swapping implementations later costs nothing if everything imports the SLF4J interface.

```java
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class OrderService {

    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public void place(Order order) {
        log.info("placing order id={} customer={}", order.getId(),
        order.getCustomerEmail());
    }
}
```

Log levels

Level	When to use
`TRACE`	Extreme detail. Off in production.
`DEBUG`	Step-by-step internals. On in dev, off in prod.
`INFO`	Significant events. Service start, request handled.
`WARN`	Something unexpected but recoverable. Retry succeeded after one failure.
`ERROR`	Operation failed. The user noticed.

Most apps run at INFO in production. WARN and ERROR should be rare enough that any of them deserves a look.

Use placeholders

```java
// Bad. Builds the string even when DEBUG is off.
log.debug("user " + userId + " requested " + count + " items");

// Good. SLF4J only builds the string if DEBUG is enabled.
log.debug("user {} requested {} items", userId, count);
```

For exceptions, pass the throwable as the last argument:

```java
try {
    db.save(order);
} catch (SQLException e) {
    log.error("failed to save order {}", order.getId(), e);
}
```

Structured logging

Plain text logs are hard to query. Structured logs ship key-value pairs that log aggregators can index.

JSON output via `logstash-logback-encoder`:

```
<!-- logback-spring.xml -->
<appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
</appender>
```

Output:

```
{
  "@timestamp": "2025-05-23T12:00:00.000Z",
  "level": "INFO",
  "logger_name": "com.example.OrderService",
  "thread_name": "http-nio-8080-exec-1",
  "message": "placing order",
  "order_id": "ord_123",
  "customer_email": "alice@example.com",
  "trace_id": "abc123def456",
  "span_id": "789xyz"
}
```

Any log aggregator (Elasticsearch, Loki, Splunk, Datadog) can index those fields and let you query `order_id:ord_123 AND level:ERROR` instead of grepping raw text.

MDC (Mapped Diagnostic Context)

A per-thread map of context that automatically gets attached to every log line. Used for correlation IDs, request IDs, user IDs, tenant IDs.

```java
import org.slf4j.MDC;

@Component
public class CorrelationFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        String correlationId = ((HttpServletRequest) req).getHeader("X-Correlation-
        ID");
        if (correlationId == null) {
            correlationId = UUID.randomUUID().toString();
        }
        MDC.put("correlation_id", correlationId);
        try {
            chain.doFilter(req, res);
        } finally {
            MDC.clear();
        }
    }
}
```

Now every log line during that request carries `correlation_id`. Trace one user's path through the system without grepping by timestamp.

MDC is thread-local. When work hops threads (async, `ExecutorService`), the context vanishes unless you propagate it. Use an MDC-aware executor or wrap your `Runnable`s.

What to log

- **Service lifecycle:** start, shutdown, config loaded.
- **Significant events:** request received (DEBUG), order placed (INFO), payment failed (ERROR).
- **External calls:** which service, latency, status code.
- **State transitions:** order moved from `PENDING` to `PAID`.
- **Errors with full context:** what we were trying to do, what failed, why.

What not to log

- **Secrets and PII.** Passwords, tokens, full card numbers, SSNs, full email when not needed.
- **Request bodies blindly.** They contain PII. Filter sensitive fields.
- **At INFO when DEBUG would do.** Noise drowns signal.
- **In tight loops without a sample.** A million log lines per second is a DoS against your own logging stack.

Metrics

Metrics are aggregated numbers over time. Counters, gauges, timers. They power dashboards and alerts.

Micrometer (the Java standard)

Spring Boot ships Micrometer. It's a vendor-neutral facade like SLF4J but for metrics. Configure once, push to Prometheus, CloudWatch, Datadog, or whatever's wired up.

```
@Component
public class OrderMetrics {

    private final Counter ordersPlaced;
    private final Counter ordersFailed;
    private final Timer placeOrderLatency;
    private final DistributionSummary orderValue;

    public OrderMetrics(MeterRegistry registry) {
        this.ordersPlaced = Counter.builder("orders.placed")
            .description("Number of orders placed")
            .register(registry);
        this.ordersFailed = Counter.builder("orders.failed")
            .description("Number of order failures")
            .register(registry);
        this.placeOrderLatency = Timer.builder("orders.place.latency")
            .register(registry);
        this.orderValue = DistributionSummary.builder("orders.value")
            .baseUnit("usd")
            .register(registry);
    }

    public void recordPlaced(Order order, Duration latency) {
        ordersPlaced.increment();
        placeOrderLatency.record(latency);
        orderValue.record(order.getTotal().doubleValue());
    }
}
```

Metric types

Type	When to use	Example
Counter	Monotonically increasing count	Orders placed, errors raised
Gauge	Point-in-time value that goes up and down	Active connections, queue depth
Timer	Duration measurements with percentiles	Request latency
DistributionSummary	Non-time measurements with percentiles	Order value, payload size

Tags (labels)

Tags let you slice metrics by dimension.

```
Counter.builder("http.requests")
    .tag("method", "GET")
    .tag("status", "200")
    .tag("endpoint", "/users")
    .register(registry);
```

In Prometheus this becomes `http_requests_total{method="GET",status="200",endpoint="/users"}`. Filter and group by any tag.

Cardinality matters

Every unique combination of tag values creates a new time series. Tags like `user_id`, `order_id`, or `email` blow up cardinality:

- 1M users * 5 endpoints * 10 status codes = 50M time series.
- Your metric store dies. Your bill explodes.

Safe tags: status code, endpoint pattern, region, instance, tenant (when bounded).

Bad tags: anything per-request (user ID, order ID, request UUID). Per-request IDs belong in logs and traces. Metric tags are for low-cardinality dimensions.

Prometheus integration

Spring Boot exposes a `/actuator/prometheus` endpoint when `micrometer-registry-prometheus` is on the classpath. Prometheus scrapes it on a schedule.

```yaml
management:
  endpoints:
    web:
      exposure:
        include: prometheus,health,info
```

Tracing

A trace is the lifecycle of a request as it moves through your services. A span is a single unit of work within that trace.

```
Trace: 8f3a2bc4...
  Span: GET /orders/123                  [0ms – 230ms]
    Span: db.query users.findById        [5ms – 15ms]
    Span: http.call payment-service      [20ms – 200ms]
      Span (in payment-service): charge  [25ms – 195ms]
        Span: db.query payments.insert   [180ms – 190ms]
```

OpenTelemetry (the standard)

OpenTelemetry (OTel) is the vendor-neutral standard. Came from the merger of OpenTracing and OpenCensus in 2019.

Two flavors:

- **Auto-instrumentation.** Attach the OTel Java agent at JVM start. It instruments common libraries (Spring, JDBC, Kafka, HTTP clients) without code changes.
- **Manual instrumentation.** Use the OTel API to add custom spans.

```java
@RestController
public class OrderController {

    private final Tracer tracer = GlobalOpenTelemetry.getTracer("order-service");

    @PostMapping("/orders")
    public Order place(@RequestBody Order order) {
        Span span = tracer.spanBuilder("place-order")
            .setAttribute("customer.id", order.getCustomerId())
            .startSpan();
        try (Scope scope = span.makeCurrent()) {
            return service.place(order);
        } catch (Exception e) {
            span.recordException(e);
            span.setStatus(StatusCode.ERROR);
            throw e;
        } finally {
            span.end();
        }
    }
}
```

Context propagation

For traces to span services, the trace ID and span ID have to travel with the request. The W3C Trace Context spec defines two headers:

```
traceparent: 00-8f3a2bc4...-789xyz-01
tracestate: vendor=value
```

OTel auto-instrumentation handles this for HTTP and gRPC clients. For custom transports (message queues, raw sockets), inject and extract context manually.

Tracing backends

Backend	Notes
Jaeger	Open source, CNCF graduated. Easy to self-host.
Tempo	Grafana's offering. Stores traces cheaply in object storage.
Zipkin	Older, simpler. Still used.
Datadog APM / New Relic / Honeycomb	Managed, full-featured. Cost grows with volume.

Sampling

Sending every trace to the backend is expensive. Sample.

- **Head-based sampling.** Decide at trace start (random N percent). Simple, can miss interesting traces.
- **Tail-based sampling.** Wait for the full trace, then decide based on what happened. Slow traces, errors, specific routes. Expensive to run but catches what matters.

Tying it all together

The point of “three pillars” is that they reinforce each other. A good observability stack lets you pivot between them.

A typical incident workflow:

1. Alert fires on a metric (error rate > 1% on `/checkout`).
2. Dashboard shows when it started and how bad.
3. Drill into one slow or failed trace.
4. The trace shows which span broke. Open its logs filtered by `trace_id`.
5. Logs reveal the exception and the offending input.

For this to work, every log line should include the current trace ID. Spring Boot 3 + Micrometer Tracing does this automatically when OTel is configured. Older setups need explicit MDC integration.

```
2025-05-23 12:00:00 INFO  [trace_id=8f3a2bc4 span_id=789xyz] OrderService - placing orde
```

SLI, SLO, SLA

The reliability framing that came from Google SRE.

Term	Meaning
SLI (Service Level Indicator)	A measurable signal of reliability. Example: success rate of `POST /orders`.
SLO (Service Level Objective)	The internal target for the SLI. Example: 99.9% success over 30 days.
SLA (Service Level Agreement)	The external commitment to customers, with penalties for missing it. Usually looser than the SLO to give engineering buffer.

Error budget = `1 - SLO`. If your SLO is 99.9%, you can fail 0.1% of requests in the window before the budget burns out.

Golden signals, RED, USE

Three different mnemonics for “what to alert on”. Pick one and stick with it.

Google’s four golden signals (for any service):

- Latency (p50, p95, p99)
- Traffic (requests per second)
- Errors (rate of failed requests)
- Saturation (how full the resource is)

RED (Tom Wilkie, for request-driven services):

- Rate
- Errors
- Duration

USE (Brendan Gregg, for resources):

- Utilization
- Saturation
- Errors

For a typical Java backend: use RED at the service level (request rate, error rate, duration), USE at the resource level (CPU, thread pools, GC).

Best practices

- Use SLF4J as the API. Pick Logback or Log4j2 as the implementation.
- Use placeholder logging. Concatenation builds the string even at log levels you’ve disabled.
- Pass exceptions to the logger as the last argument. Skip `log.error(e.getMessage())`.
- Structured JSON logs in production. Plain text in local dev.
- Set up MDC with a correlation ID filter for HTTP and a context propagator for async work.
- Keep log levels honest. INFO is for things you want to see in steady state.
- Cap metric cardinality. Per-request IDs go in logs and traces. Metric tags are for low-cardinality dimensions.
- Sample traces. 100% sampling at production volume is rarely sustainable.
- Include trace ID in every log line. The pivot from metrics to traces to logs depends on it.
- Define SLOs before you write alerts. Without an SLO, alerts are based on vibes.
- Alert on user-visible symptoms like error rate and latency. Internal metrics like CPU belong on dashboards.

Common gotchas

- **String concatenation in log statements.** `log.debug("x=" + x)` builds the string even when DEBUG is off. Use `log.debug("x={}", x)`.
- **MDC across threads.** Spawning a new thread loses the context. Use an MDC-aware `Runnable` wrapper or `TaskDecorator` in Spring.
- **Logging secrets.** `log.info("user={}", user)` where `user.toString()` includes the password hash. Override `toString` carefully or use field-level filters.
- **High-cardinality metric labels.** `tag("user_id", id)` with a million users equals a million time series. Cost goes up, query gets slow, eventually metrics break.
- **Counting with gauges or gauging with counters.** Counters only go up. Gauges go up and down. Pick the right type.
- **Alerting on raw CPU instead of saturation.** 80% CPU on a 4-core box at peak is fine. 80% with a long run-queue is a problem.
- **No trace ID in logs.** You see the slow trace in Jaeger but can't find the matching log lines. Include `trace_id` in every log line.
- **Tail-based sampling without buffer.** If the collector buffer is too small, late spans get dropped before the decision fires.
- **One Logger per instance instead of per class.** `LoggerFactory.getLogger(this.getClass())` works but `static final` per class is faster and idiomatic.
- **Synchronous logging in hot paths.** Logback's `AsyncAppender` or Log4j2's async loggers move the I/O off the request thread.
- **Treating logs as the primary debugging tool in distributed systems.** Without traces, you'll spend hours stitching logs across services. Traces give you the path for free.

Quick reference

Concept	Key fact
SLF4J	The logging facade for Java
Logback / Log4j2	The common implementations
Log levels	TRACE < DEBUG < INFO < WARN < ERROR
Placeholder logging	`log.info("x={}", x)`
MDC	Per-thread context map, used for correlation IDs
Structured logs	JSON output via `logstash-logback-encoder`
Counter	Goes up only. Errors, requests.
Gauge	Goes up and down. Queue depth, active connections.
Timer	Latency with percentiles
Micrometer	The Java metrics facade. Spring Boot default.
OpenTelemetry	The vendor-neutral observability standard
Trace	The lifecycle of a request across services
Span	A single unit of work within a trace
W3C `traceparent`	The HTTP header that carries trace context
SLI / SLO / SLA	Indicator, internal target, external commitment
Golden signals	Latency, traffic, errors, saturation
RED	Rate, errors, duration
USE	Utilization, saturation, errors